

Handoff — 2026-08-03

Branch `main` , **pushed, commit** `e2cfa16` . **Suite 58 passed, 0 failed.** The `refit-and-semantics-guard` branch was merged fast-forward and is gone; work from `main` .

Read `docs/LOOKAHEAD-design.md` first. It is where the session ended up and the only thing here with a live decision attached.

1. THE SHORT VERSION

Three of the four roadmap items handed over on 2026-08-02 **do not exist**. Two real bugs were found and fixed. Two features shipped and are honestly labelled as unproven. One measurement — the last one of the night — is forty times larger than anything else and points somewhere new.

PORYGON2 retrained on a corrected bot	+0.11 accuracy points
per-Pokemon HP distribution added to the value function	+0.03
one turn of foresight	+4.91

Everything the project has been doing is in the first two rows.

2. WHAT WAS WITHDRAWN, AND DO NOT RE-INHERIT IT

Each died to a measurement. The measurements are committed; re-run them rather than trusting this list.

- **"Greedy over-clicks conditional moves."** The 47.9% / 33.2% / 33.9% triple came from two corpora differing on **three** levers at once — greedy, joint, AND the weights file. On a single-lever paired corpus the gap is **-0.7 points** against a *measured* known-zero band of ± 7 . Tool: `engine/conditional_audit.js` , which derives the 39 conditional moves from the dex's own handlers and carries `--calibrate` so its interval is checked against a known zero rather than trusted.
 - **"MAG under-switches."** The lever was **off**. Voluntary switches in every self-play corpus on disk were exactly `4 x games` — the opening send-outs and nothing else. The `partial_bring` evidence was circular (bring count and switch count are the same quantity), and that filter is a *decisiveness* filter: the short side is the WINNER in **1,008 of 1,008** games.
 - **"Switching would pay if turned on."** It does not, in the config that ships. **45.9% of 874 decisive pairs, DECIDED against.** An earlier run said 54.1% of 1,032 and decided FOR — that run had joint off and the pre-refit vector. `showdown_bot.js` is unchanged and its comment refusing to enable switching was right.
 - **"The Protect excess is in the weights."** It is not. `engine/protect_calibration.js` scores the shipped vector against the human decisions it was fitted on: **19.26% predicted, 18.86% observed, ratio 1.021.**
 - **"PORYGON2's ceiling is contaminated training data."** Rebuilt on a greedy, open-sheet, correct-ability corpus: **63.59% -> 63.70%**. The ceiling is real. PORYZ's premise is now well-evidenced instead of assumed.
-

3. WHAT IS ACTUALLY WRONG WITH PROTECT, because it was the one real item

Two mechanisms with **opposite signs**, which is why the aggregate reads 1.021 and looks calibrated. Will named the unifying cause: *"ITS PART OF THE MULTI TURN TAILWIND TYPE ANALYSIS."*

$\text{corr}(\text{Protect residual, MULTI-TURN moves on the sheet}) = -0.343$ $\text{corr}(\text{Protect residual, INSTANT status moves}) = +0.066$ (nothing) 0 / 1 / 2 multi-turn moves -> +0.8 / -1.3 / -4.9 points

The model **under**-predicts Protect where a passed turn accrues value (Blaziken -18.0 on Speed Boost; Gengar -19.0 burning Perish turns behind Shadow Tag) and **over**-predicts it where a better multi-turn investment sits on the same sheet (Pelipper, Politoed, Farigiraf: Tailwind, Trick Room, screens). Instant status moves — Follow Me, Fake Out, Taunt — carry **none** of it.

Ruled out and recorded so nobody re-inherits them: `priorLogP` underweighting (corr 0.032 with the residual), a population mismatch in `move-priors.json` (it matches the fit corpus to three decimals), Choice-lock (5 of 11,517 decisions), and the mega ability gap — fixed universally this session, and **the residuals did not move a decimal place**.

4. THE TWO REAL BUGS

The mega identity gap, fixed universally. A team sheet lists the **pre-mega** ability, so `mon.ability` said Cursed Body for a Mega Gengar with Shadow Tag. `buildMon('mawile-mega')` already returned "Huge Power" correctly — and `board.js` then **overwrote it with the sheet's "Intimidate"**. Huge Power doubles Attack. **23,788 of 94,476 sheet slots (25.2%)** carry a mega stone whose forme ability differs from the sheet's.

Fixed with **one resolver for the whole identity** — `board.js effective(mon, dex)` returns species, ability, types, base stats and weight together, so a caller cannot hold the mega's ability beside the base forme's types. Nothing is named; the ability derives from the dex's own `isMega` + `abilities`, and `tests/test-effective-identity.js` proves it reproduces **all 62** checkable entries of `medicham2-browser.js`'s hand-written `MEGA_ABIL` map. That map is now provably redundant.

The fixture could not see any of this — all 74 feature hashes were byte-identical across the change, because no mega with a differing ability stands on its boards. That is why the guard is a ratcheted fitness function (**234 raw reads across 40 files, may only shrink**) rather than another fixture entry: the next regression is a different species.

Two new features were node-only. `board-data.js` carries an explicit allow-list and neither ability `onResidual` nor condition `duration` was on it, so the browser scored both as always-zero while node did not. **58 of 58 features now agree to 6 dp**, up from 56 of 58.

5. WHAT SHIPPED

- `setupTurns` **+0.777** and `passTurnAccrues` **+1.129** (ranks 21 and 13 of 58). Derived: turns beyond the one you spent, from the dex's own duration fields; and a Protect-family click while the user's *effective* ability gains on residual. **The H2H is UNDECIDED at 51.8% of 674 decisive pairs.** Better at predicting human clicks, not yet better at winning — the expected shape, since a greedy imitation policy maximises click-likelihood. Converting it needs the outcome objective (`train_policy.js`, MACHAMP), not another imitation refit.
- `engine/player_digest.js` — which bot made any data, hashing feature semantics + pair features

- damage table + both weight vectors. Content, not a commit. `mew.js` stamps it into every corpus. It immediately caught the July Smogon priors moving 16 features under a vector fitted an hour earlier.
- `fit_policy.js` `HOLDOUT_SINCE` / `TRAIN_SINCE` — a TIME split, because the random-by-game split cannot see staleness by construction.
- **July Smogon data** (15.76M -> 23.64M battles). Both vectors refitted; the new features held across the swap (+1.127 -> +1.129, +0.765 -> +0.777), which matters more than the fit itself.

6. THE RECENCY ANSWER, because it was asked and is now settled

Three streams had three unchosen policies. Measured on the same August test set:

train June+July (185,644 decisions) held out on August -1.7218 train July only (108,156 decisions) held out on August -1.7216

Dropping June costs nothing and gains nothing. No staleness to decay away at three months' depth — which argues for **keeping** it, since `passTurnAccrues` fires on 0.4% of candidates and discarding 42% of the data for zero gain makes rare features worse for free. Standard: **no decay on our replays, newest month for Smogon.** Re-measurable when the corpus spans six months.

7. WHERE TO START, IN ORDER

1. **Fix the** Infinite loop **in** `engine/lookahead_cost.js` . 25-40% of forks throw it. Hypothesis, untested: a restored battle is not in `requestState === 'move'`, so `choose('default')` never advances it and the engine spins — i.e. my driving, not `fromJSON` . Must reach ~0 before G3 means anything, because a search cannot evaluate a cell it cannot simulate and the value of a hole is unknown, not zero.
2. **Measure the top-3 truncation rate.** `fit_joint.js` already computes it for top-6 (**11.3%**); same replay, different K, no new code. It **caps G3** — a search cannot recover value from a branch it never enumerated. And G2 says top-6 is unaffordable on every board measured (cost per fork ranges **3.5 to 32.2 ms**, a factor of nine).
3. **G3** — replace the oracle with a real matrix-game evaluation and measure what fraction of +4.91 survives. Expect well under half.
4. **G4** — SPRT against the current greedy player, read as it goes.

Do not add more MAG features. Two rounds tonight produced +0.03 and +0.11 and an undecided H2H.

8. THE CEILING, SINCE IT WAS ASKED DIRECTLY

Will: *"I DON'T WANT TOP 10% I WANT A SOLVER THAT WILL ALWAYS WIN."*

That is unreachable, and this repository already measured why. `MODELS.md:230` : the 55% skill ceiling was bot-contaminated; **clean it is 52.4%, and the interval includes 50.** In Reg M-B the higher-rated player barely beats a coin. Damage rolls, accuracy, criticals, speed ties and simultaneous commitment cap how much skill can express itself per game.

Deeper: a simultaneous-move game with a non-transitive component has its optimum in **mixed** strategies, which lose some fraction of the time by construction. Perfect against perfect in a symmetric matchup is 50%. Nash buys **unexploitable**, not unbeatable — `POKER-TO-POKEMON.md` §6.3 already says it.

Where "always wins" IS literally achievable: DUSK , the endgame exact solver ($\leq 2v2$, $1v1$). Small enough subgames are solvable and perfect play is computable, so it does always win when a win exists. It is on the roadmap, unbuilt, and it is the only component where the word "solver" is accurate.

The realistic targets are: **unexploitable** globally (SLOWKING/Nash), **maximally exploitative** against a read opponent (HYPNO, unbuilt), and **perfect** in solved endgames (DUSK, unbuilt).

9. HOW TO WORK, LEARNED THE HARD WAY TONIGHT

- **Measure the noise floor before believing an effect.** Split one arm's games in half, where the truth is 0.00 by construction, and see how big a difference the instrument invents. That killed three claims in minutes. `conditional_audit.js --calibrate` does it.
 - **Check the corpus stamps before attributing an effect to a lever.** The 47.9% came from corpora differing in three ways. This was the single most common failure of the day.
 - **A lever reading zero may never have been switched on.** Voluntary switches were exactly 4 x games in every corpus.
 - **The guards work — let them.** They caught me four separate times (a raw MC.mons index, three silent catches, a redundant raw read, a swallowed parse error). Fixing beats re-baselining: **silent catches went 239 -> 232, the first time that number has gone down.**
 - **My predictions were wrong and my measurements were right,** consistently. The ability fix would shrink Blaziken's residual (it did not move a decimal). Contaminated data explained PORYGON2's ceiling (it did not). G2 passed at 40/40 (that was one lucky board; it is 25-40% failures).
-

10. HOUSEKEEPING

- `SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown` still required.
- `engine/fit_joint.js` and `engine/fit_policy.js` both want `node --max-old-space-size=4096`.
- **Never** `git add -A` — the ingest churns `archetypes.json`, `kad-replays.js`, `live.js`, `conformance.json`.
- **A refit invalidates seven artifacts** — `counterplay`, `winrate-backtest`, `opponent-calibration`, `weight-multiplicity`, then the `mag / mew / scoreboard` bundles. `engine/provenance.js` names them; regenerate in that order, then re-run it to prove nothing trails.
- `engine/artifact_audit.js` compares **mtimes**, so a `git checkout` can fire it spuriously — all three files land within milliseconds of each other. Check B (content) is the load-bearing one.
- Watched numbers: **6 declared misses** (`grep mayMiss:`), **232 silent catches** (`data/silent-catch-baseline.json`), **234 raw identity reads** (`data/effective-identity-baseline.json`), **16 hand-rolled lookups** (`data/mc-key-baseline.json`). All four may only shrink.